KVL Live Translator

Ye app kaise kaam karta hai - poori guide (Hinglish me)

Is document me hum step by step samjhenge ki app kya karta hai, andar se kaise bana hai, ek meeting ke dauran audio se lekar translation tak data kaise flow karta hai, har feature (15 phases) kya deta hai, ise chalaya kaise jata hai, aur agar kuch kharab ho to kahan dekhna hai.

Kisne kya padhna chahiye:

- **Naye user** - Section 1, 2, 5 aur 8 padho.
- **Developer** - Section 3, 4, 6, 7 aur 9 zaroor dekho.
- **Problem aa rahi hai** - seedha Section 9 (Troubleshooting) par jao.

Ye guide project ke README.md, CHANGELOG.md aur source code par based hai. Jahan cheez abhi tak verify nahi hui, wahan saaf likha hai.

1. App kya hai?

KVL Live Translator ek **real-time meeting language assistant** hai. Maan lo aap Zoom ya Google Meet par English me meeting kar rahe ho aur English samajhne me dikkat hoti hai. Ye app:

- Meeting ki English awaaz **sunta** hai (aapke microphone se aur/ya computer ki system audio se).
- Use **text me likhta** hai (Speech-to-Text), timestamp aur confidence ke saath.
- Us text ko **Hindi / Hinglish / Simple Hindi / Simple English** me translate karta hai.
- Aap Hindi ya Hinglish me bolo, to usse **professional English reply** ka draft banata hai.
- Zaroorat ho to reply ko **awaaz me bol** bhi sakta hai (Text-to-Speech).
- Meeting ke dauran **questions, decisions, action items, deadlines** apne aap pakadta hai (Copilot).
- Meeting ke baad ek **11-section ka summary** banata hai, jise PDF ya text me export kar sakte ho.

Ye ek **desktop app** hai (Electron + React + TypeScript), aur iska backend aapke apne computer par chalta hai. Raw audio kabhi disk par save nahi hota.

2. Ek meeting me data ka safar (big picture)

Sabse zaroori cheez samajhna: data ek line me aage badhta hai, aur har step ke paas ek ya zyada **fallback** hota hai. Agar ek tareeka fail ho jaye, to agla try hota hai.

```
Microphone / System audio
       |
       v
[Renderer] Audio capture + VAD (bolne wale hisse ko kaatna)
       |   chunk:ready  ->  audio:chunk (WebSocket)
       v
[Backend] SpeechManager -> SpeechProviderRouter
             1. Local Whisper (whisper.cpp, offline)
             2. Cloud STT (OpenAI-compatible)
       |   transcript:update  (screen par English text)
       v
[Backend] TranslationManager -> TranslationRouter
             1. Local phrasebook   2. Argos Translate
             3. LibreTranslate     4. Cloud LLM
       |   translation:update (screen par Hindi/Hinglish)
       v
SQLite me save  ->  History / Copilot / Summary
```

Seedhi bhasha me: aap bolte ho, app chhote-chhote tukdon me awaaz kaat leta hai, har tukde ko text me badalta hai, phir text ko translate karta hai, aur dono panels (LIVE ENGLISH aur HINDI-HINGLISH) me live dikhata hai. Saath hi sab kuch aapke computer ke database me save hota hai.

3. Andar se app kaise bana hai (architecture)

Project ek **pnpm monorepo** hai - ek hi folder me kai chhote packages aur do apps.

Folder	Kaam
apps/desktop	Electron app. Isme main process (window, shortcuts, backend start/stop), preload (safe bridge) aur renderer (React UI + audio engine) hain.

Folder	Kaam
apps/backend	Plain Node.js + TypeScript server. WebSocket server, SQLite database, STT, translation, copilot, summary sab yahin hai.
packages/shared	Dono taraf ke common types aur WebSocket protocol.
packages/ui	Design system - Platinum Silver & Gold theme aur React components.
packages/audio, speech, translation, tts, response, explain, copilot, summary	Har feature ka interface (contract). Asli implementation zyadatar apps/backend ya apps/desktop me hai.

Kuch design faisle jo jaanne layak hain:

- **Renderer backend se WebSocket par baat karta hai**, Electron IPC par nahi. Isse backend kal ko Electron ke bahar bhi use ho sakta hai.
- **SQLite (better-sqlite3) sirf backend me** hai, jo normal system Node me chalta hai. Isse native module Electron ke Node version se nahi takrata.
- **Audio sirf renderer me** process hota hai, kyunki getUserMedia aur AudioWorklet browser-side APIs hain. Backend ko sirf chhote audio chunks milte hain.
- **Har WebSocket connection par token** lagta hai. Har launch par naya 256-bit token banta hai, taaki computer ka koi aur program meeting ka content na padh sake.

4. App start hone par kya hota hai

Jab aap `pnpm dev` chalate ho:

- electron-vite main aur preload code build karta hai.
- Renderer ke liye Vite dev server `http://localhost:5173` par chalta hai.
- Electron khulta hai. Main process ek **naya random token** banata hai aur backend ko (`pnpm --filter @kvl/backend run dev` se) start karta hai.
- Main process backend ke WebSocket (`ws://127.0.0.1:4319`) par baar-baar try karta hai (har 0.5 sec, 20 baar tak). Jab connect ho jata hai tabhi window khulti hai. Agar 10 sec me nahi hua, to app band ho jata hai.
- Window khulne par renderer preload ke zariye port aur token leta hai, aur connect karta hai. Top bar me **Connected** dikhta hai.

Agar backend achanak band ho jaye to main process use apne aap **dobara start** karta hai (maximum 5 baar, har baar thoda zyada wait karke).

5. Features - 15 phases ek nazar me

Phase	Kya milta hai
1	Architecture aur poora navigable UI shell (Dashboard, Meeting Translator, Settings, History, API Providers, Audio Diagnostics, About).
2	Audio capture: microphone + system audio, buffering, Voice Activity Detection (VAD). Test Mode me bina mic ke bhi dummy tone se test ho sakta hai.
3	Speech-to-Text: local whisper.cpp, fail ho to cloud. Language auto-detect hoti hai.
4	Translation: Hindi, Hinglish, Simple Hindi, Simple English. Business terms (API, sprint, backend...) translate nahi hote, jaise ke taise rehte hain.
5	Live Translator UI: do panels, status bar, font size, auto-scroll, compact mode, floating mini window. Pause Translation (audio chalta rahta hai, sirf translation ruki rehti hai).
6	My Response: aap Hindi/Hinglish me bolo ya likho, app professional English reply draft banata hai.
7	Text-to-Speech: reply ko bolna. Local (espeak-ng), phir browser, phir cloud.
8	'What does this mean?': kisi bhi sentence par click karo, uska matlab aur context samjhata hai. History me save hota hai.
9	Smart Meeting Copilot: question, decision, action item, deadline, requirement, objection, pricing, follow-up jaise events apne aap pakadta hai.
10	AI Meeting Summary: 11 sections. 7 sections sirf Copilot ke confirmed facts se bante hain (AI ke bina), 4 sections me LLM synthesis hai. Meeting History aur export.
11	Cost control: economy / balanced / quality mode, monthly budget limit, Local-Only Mode, provider health tracking, request cache.
12	Meeting window: always-on-top, opacity, click-through, 10 global keyboard shortcuts.
13	Privacy aur security: WebSocket token, API keys encrypted, transcript retention setting, logs me secrets redact.
14	Stability: lambi meeting me memory bounded, crash recovery (agli baar app khule to adhuri session recover/delete ka option), mic auto-retry.
15	Final release: 10-step first-run setup wizard, installer packaging.

6. Har step andar se - detail me

6.1 Audio capture (Phase 2)

- **MicrophoneCapture** aur **SystemAudioCapture** raw audio stream kholte hain.
- Ek **AudioWorklet** audio ko chhote frames me todta hai.
- **AudioBuffer** ek bounded ring buffer hai - memory kabhi bahut nahi badhti.
- **VAD** (energy/RMS + hysteresis, koi ML nahi) pehchanta hai ki kab aap bol rahe ho aur kab chup ho. Sirf bolne wale hisse chunk bante hain.
- Chunk tayyar hone par `chunk:ready` event nikalta hai, jise STT engine sunta hai.

OS	System audio support
Windows	Bharosemand (WASAPI loopback).

OS	System audio support
macOS	Direct nahi. BlackHole jaisa virtual audio device lagao aur use microphone ki tarah chuno.
Linux	PipeWire/PulseAudio par depend karta hai. Microphone dropdown me output ka '.monitor' source chuno.

Privacy: raw audio disk par kabhi save nahi hota. Memory me rehta hai aur chunk ban jane ke baad hat jata hai.

6.2 Speech-to-Text (Phase 3)

Chunk base64 PCM16 ke roop me WebSocket se backend jata hai. **SpeechProviderRouter** pehle local whisper.cpp try karta hai; wo na ho ya fail ho jaye to cloud provider. Dono language khud detect karte hain (English hard-code nahi). Output **TranscriptBuffer** me startMs ke hisaab se sahi order me lagta hai, chahe chunks aage-peeche pahunche ho.

Local Whisper chalane ke liye apps/backend/.env me do cheezein set karo:

```
WHISPER_CPP_BINARY_PATH=/path/to/whisper-cli
WHISPER_CPP_MODEL_PATH=/path/to/ggml-tiny.en.bin
```

Cloud STT ke liye API Providers screen par provider add karo (kind: Speech-to-Text, vendor: openai ya koi bhi OpenAI-compatible endpoint) aur API key daalo. Restart ki zaroorat nahi.

6.3 Translation (Phase 4)

TranslationRouter priority order me providers try karta hai, aur jo unavailable, unsuitable, rate-limited ya low-confidence ho use skip karta hai:

#	Provider	Note
1	Local phrasebook	Hamesha available, offline, lekin sirf jaane-pehchane phrases.
2	Argos Translate	Local Python NMT. Install karna padta hai (pip install argostranslate + en-hi package).
3	LibreTranslate	Free REST API ya self-hosted (LIBRETRANSLATE_BASE_URL).
4	Cloud LLM	Paid. **Sirf yehi asli Hinglish aur Simple modes kar sakta hai.**

Hinglish ka sach: Argos/LibreTranslate/phrasebook jaise engines sirf standard Hindi (Devanagari) dete hain. Casual Romanized Hinglish ek instruction-following kaam hai, translation nahi. Isliye in engines ka 'hinglish' mode Hindi ko Roman me transliterate karke approximate karta hai - asli code-mixing nahi. Asli Hinglish aur Simple modes ke liye cloud LLM chahiye.

Business terms: translation se pehle API, sprint, deployment jaise terms placeholder token se badal diye jate hain aur baad me wapas lag jate hain, taaki koi provider unhe galat translate na kare.

Rate limits: har provider ka apna health tracker hai - default 20 requests/minute ki self-imposed limit, rate-limit cooldown, aur lagatar errors par circuit breaker.

6.4 My Response aur Text-to-Speech (Phase 6, 7)

Aap Hindi/Hinglish me bolte ho ya likhte ho, app use ek professional English reply me badalta hai (style choose kar sakte ho). Phir chaho to us reply ko bulwa sakte ho: pehle espeak-ng (local), phir browser ki Web Speech, phir cloud TTS. Virtual audio cable ke zariye ye awaaz meeting me bhi bheji ja sakti hai (app me iski guidance di gayi hai).

6.5 Copilot aur Summary (Phase 9, 10)

Copilot transcript ko live dekhta hai. Ek free local detector aur batched cloud LLM analysis dono milkar questions, decisions, action items, deadlines, requirements, objections, pricing, technical requirements, follow-ups aur client requests pakadte hain.

Summary 11 sections ka hota hai. 7 sections poori tarah Copilot ke confirmed facts se bante hain - isme AI andaza nahi lagata, isliye ye structurally accurate hain. Baaki 4 sections me LLM ki real synthesis hoti hai. Meeting History screen se export (PDF/text) aur delete ho sakta hai.

6.6 Cost aur Providers (Phase 11)

- **Cost modes:** economy, balanced, quality.
- **Monthly budget:** hard limit - paid providers us se aage nahi chalte.
- **Local-Only Mode:** koi bhi cheez internet par nahi jati.
- **Request cache:** same request dobara ho to paisa nahi lagta; cache ka size bounded hai.

7. Safety aur privacy

- WebSocket token har launch par naya, sirf memory me - disk par kabhi nahi likha jata aur logs me nahi aata.
- Token ki comparison constant-time hoti hai (timing attack se bachne ke liye).
- Provider API keys database me **encrypted** rehti hain. Backend ke paas ek transient in-memory copy hoti hai.
- Logs me API key, token, secret, password ke pattern redact hote hain.
- Transcript retention configurable hai, aur Settings me 'Delete Session Data' se saara local data hata sakte ho.
- Raw meeting audio kabhi save nahi hota.

8. App chalane ka tareeka

Pehli baar setup

```
corepack enable
corepack prepare pnpm@11.22.0 --activate
pnpm install

cp .env.example .env
cp apps/backend/.env.example apps/backend/.env
cp apps/desktop/.env.example apps/desktop/.env
```

Requirements: Node.js 20+, aur better-sqlite3 ke liye python3, make, g++. **Project ka path space wala nahi hona chahiye**, warna node-gyp ke build scripts toot jate hain.

Roz ka kaam

```
pnpm dev            # poora desktop app (backend khud start ho jata hai)
pnpm dev:backend    # sirf backend (ws://127.0.0.1:4319)
pnpm build          # typecheck + backend build + desktop build
pnpm typecheck      # saare packages ka type check
```

Dhyan rakho: `pnpm dev` aur `pnpm dev:backend` ek saath mat chalao. Dono port 4319 maangte hain; doosra fail ho jata hai, aur desktop app readiness timeout par band ho jata hai.

Pehli baar app khulne par

10-step setup wizard aata hai: audio setup, microphone test, system audio test, language, provider, privacy, keyboard shortcuts, aur ek real test meeting. Mic test me meter hilna chahiye.

Tests

Backend ke tests alag-alag chalte hain, jaise `pnpm --filter @kvl/backend test:speech-core` (aur test:translation-core, test:response-core, test:tts-core, test:explain-core, test:copilot-core, test:summary-core, test:cost-core, test:security-core). Desktop ke liye test:audio-core, test:desktop-main-core, test:meeting-store-core. Abhi koi ek `pnpm test` nahi hai.

9. Troubleshooting - kuch kharab ho to

Problem	Wajah	Kya karo
Backend 'Disconnected' dikhta hai, aur log me 'rejecting connection: invalid or missing auth token'	Preload script load nahi hua (build index.mjs banata hai, code index.js dhundta tha), isliye renderer ko asli token nahi mila.	Ye fix ho chuka hai (main/index.ts aur miniWindow.ts me .mjs). Fix ke baad app restart karo.
Desktop app start hote hi band ho jata hai; log me 'did not become ready in time'	Port 4319 pehle se kisi aur backend ne pakda hua hai (EADDRINUSE).	`ss -ltnp | grep 4319` se process dhundo aur kill karo. pnpm dev:backend band rakho.
Linux (Wayland) par window banne ke baad app band ho jata hai	Native Wayland par Electron ka main process exit ho jata tha.	pnpm dev ab --ozone-platform=x11 deta hai. Packaged build ko khud us flag ke saath chalao.
Speech recognition failed toast	Local whisper timeout ya fail hua, aur cloud STT configured nahi hai.	whisper ke paths .env me check karo, ya API Providers me cloud STT key daalo.
Translation failed toast	Saare providers ya to low-confidence the ya unavailable (jaise Argos install nahi).	Argos install karo, ya cloud translation key daalo. Pehle STT sahi karo.
'FOREIGN KEY constraint failed' (transcript persist)	Segment ke session id ka row database me nahi hai - jaise wizard ke mic test me, jahan asli session nahi banta. (Ye abhi andaza hai, poori tarah verify nahi hua.)	Asli session Start karke test karo. Ye bug ki tarah investigate karna baki hai.
System audio nahi aati	OS limitation (Section 6.1 ki table dekho).	Audio Diagnostics screen par 'Test System Audio' chalao, aur virtual device ya .monitor source use karo.

10. Abhi ke known gaps (honest status)

- **Git:** pehle koi commit nahi tha. Ab initial commit aur kuch fixes ho chuke hain.
- **Lint:** root me script hai, par kisi package me lint define nahi, isliye pnpm lint kuch check nahi karta.
- **Tests kam hain:** sirf backend aur desktop ke kuch core tests hain; packages/* me koi nahi.
- **Update check:** apps/desktop/src/main/updateCheck.ts me real GitHub Releases URL abhi placeholder hai.
- **Code signing / notarization / update server** abhi configured nahi hain.
- **Windows aur macOS installer** test nahi hue; sirf Linux par build verify hua hai.
- **Real-hardware checks pending:** README ke hisaab se real mic, system audio, device disconnect/reconnect aur lambi meeting ki stability aapki machine par verify karni baki hai.

11. Chhota glossary

Shabd	Matlab
STT	Speech-to-Text - awaaz se text.
TTS	Text-to-Speech - text se awaaz.
VAD	Voice Activity Detection - pehchanna ki kab koi bol raha hai.
Chunk	Bolne wale hisse ka ek chhota audio tukda.
Provider	Koi bhi service jo ek kaam kare - jaise whisper.cpp, Argos, OpenAI.
Fallback	Pehla provider fail ho to agla try karna.
Hinglish	Hindi ko Roman (English) akshar me likhna, jaise 'aap kaise ho'.
Renderer / Main / Preload	Electron ke teen hisse: UI, system-level control, aur dono ke beech ka safe bridge.
WebSocket	Renderer aur backend ke beech lagatar khuli rehne wali connection.